Assignment 2 Report

Implementation of Banker’ s Algorithm for Deadlock Avoidance

1. Introduction

Deadlock is a situation in an operating system where a group of processes are blocked because each process is holding a resource and waiting for another resource held by another process. Banker’s Algorithm is a deadlock avoidance algorithm that ensures the system always remains in a safe state by checking resource allocation before execution.

The main objective of this assignment is to implement Banker’s Algorithm using Python and determine whether the system is in a safe state or unsafe state and find the safe sequence.

---

2. Objectives

- To understand deadlock and deadlock avoidance.
- To implement Banker’s Algorithm.
- To calculate Need Matrix.
- To determine Safe Sequence.
- To check whether the system is in safe or unsafe state.

---

3. Tools and Technology Used

- Python 3.x
- Built-in Python libraries
- Windows/Linux OS
- GitHub for version control

---

4. System Model

The system consists of:

- Processes
- Resources
- Allocation Matrix
- Maximum Matrix
- Available Resources
- Need Matrix

Formula Used:

Need = Maximum – Allocation

The Need matrix shows the remaining resources required by each process to complete execution.

---

5. Algorithm Explanation (Banker’ s Algorithm)

Steps of Banker’s Algorithm:

1. Input Allocation Matrix.
2. Input Maximum Matrix.
3. Input Available Resources.
4. Calculate Need Matrix using Need = Max – Allocation.
5. Initialize:
    - Work = Available
    - Finish = False for all processes
6. Find a process such that Need ≤ Work.
7. If found, add its allocated resources to Work.
8. Mark process as finished.
9. Repeat until all processes finish.
10. If all processes finish → Safe State, otherwise Unsafe State.

---

6. Output Explanation

The program displays:

- Allocation Matrix
- Maximum Matrix
- Need Matrix
- Available Resources
- Safe Sequence
- System Safe or Unsafe

If the safe sequence exists, then the system is in a safe state and deadlock will not occur.

---

7. Result Analysis

After running the program, the Need Matrix was calculated and the safety algorithm was applied. The program checked whether each process could be executed with the

available resources. The system was found to be in a safe state and the safe sequence was generated successfully.

This shows that the Banker's Algorithm successfully avoids deadlock by allocating resources safely.

---

8. Conclusion

In this assignment, we implemented Banker's Algorithm using Python. We calculated the Need Matrix and applied the Safety Algorithm to determine whether the system is in a safe state or unsafe state. The safe sequence was generated and the result was analyzed.

Banker's Algorithm is useful for deadlock avoidance because it ensures that the system never enters an unsafe state.

---

9. References

- Operating System Concepts Book
- Classroom Notes
- Python Documentation